Case 3 - Scripting

Python script using Pandas | Containerized with Docker

1. Python Script (case3.py)

```
import pandas as pd

# ============================================================
# Case 3: Money Flow Index (MFI) Calculation
# Input : sample.csv
# Output: sample_output_5.csv, sample_output_7.csv, sample_output_10.csv
# ============================================================

# ------------------------------------------------------------
# STEP 1: Load the dataset
# ------------------------------------------------------------
df = pd.read_csv('sample.csv')

# ------------------------------------------------------------
# STEP 2: Calculate Typical Price
# Typical Price = average of High, Low, and Close for each day
# ------------------------------------------------------------
df['Typical_Price'] = (df['High'] + df['Low'] + df['Close']) / 3

# ------------------------------------------------------------
# STEP 3: Calculate Money Flow
# Money Flow = Typical Price * Volume
# The value is POSITIVE if today's Typical Price > yesterday's
# The value is NEGATIVE if today's Typical Price <= yesterday's
# The first row has no Money Flow (no previous day to compare)
# ------------------------------------------------------------
df['Money_Flow'] = None

for i in range(1, len(df)):
    money_flow = df.loc[i, 'Typical_Price'] * df.loc[i, 'Volume']
    if df.loc[i, 'Typical_Price'] > df.loc[i - 1, 'Typical_Price']:
        df.loc[i, 'Money_Flow'] = money_flow
    else:
        df.loc[i, 'Money_Flow'] = -money_flow

# ------------------------------------------------------------
# STEP 4: Define function to calculate MFI for a given m
#
# Parameters:
#   m (int): lookback window, must be between 2 and len(df)
#
# For each row i (starting from index m, i.e. the m+1th row):
#   - Look back at the last m rows (index i-m to i-1)
#   - Sum of Positive Money Flow = sum of Money_Flow values > 0
#   - Sum of Negative Money Flow = sum of absolute Money_Flow values < 0
#   - MFI = 100 * (SPMF / SNMF) / (1 + (SPMF / SNMF))
#
# Note: first m rows will have no MFI value (NaN)
# Note: Money_Flow at index 0 is None (no previous day),
#       so we skip it when summing. This is handled naturally
#       since None/NaN values are ignored by sum().
# ------------------------------------------------------------
def calculate_mfi(df, m):
    # Validate m is within acceptable range
    if m < 2 or m > len(df):
        raise ValueError(f"m must be between 2 and {len(df)}, got {m}")

    # Copy df to avoid modifying the original
    df_result = df.copy()

    # Initialize new columns with None
    df_result['Positive_Money_Flow_Sum'] = None
    df_result['Negative_Money_Flow_Sum'] = None
```

```
    df_result['MFI'] = None

    for i in range(m, len(df_result)):
        # Get the last m rows of Money_Flow
        window = df_result['Money_Flow'].iloc[i - m:i]

        # Sum of positive money flows in the window
        positive_sum = window[window > 0].sum()

        # Sum of negative money flows (use absolute value)
        negative_sum = abs(window[window < 0].sum())

        # Store sums
        df_result.loc[df_result.index[i], 'Positive_Money_Flow_Sum'] = positive_sum
        df_result.loc[df_result.index[i], 'Negative_Money_Flow_Sum'] = negative_sum

        # Calculate MFI
        # Guard against division by zero: if negative_sum is 0,
        # MFI is set to 100 (all positive flows, maximum value)
        if negative_sum == 0:
            df_result.loc[df_result.index[i], 'MFI'] = 100.0
        else:
            money_ratio = positive_sum / negative_sum
            df_result.loc[df_result.index[i], 'MFI'] = 100 * money_ratio / (1 + money_ratio)

    return df_result

# ------------------------------------------------------------
# STEP 5: Generate output CSVs for m=5, m=7, m=10
# ------------------------------------------------------------
for m in [5, 7, 10]:
    df_output = calculate_mfi(df, m)
    output_filename = f'sample_output_{m}.csv'
    df_output.to_csv(output_filename, index=False)
    print(f"Generated {output_filename}")

print("All done.")
```

2. Dockerfile

```
# Use official Python slim image to keep the image size small
FROM python:3.11-slim

# Set working directory inside the container
WORKDIR /app

# Copy the script and input data into the container
COPY case3.py .
COPY sample.csv .

# Install required Python library
RUN pip install pandas

# Run the script when the container starts
CMD ["python", "case3.py"]
```

3. How to Run Inside Docker

```
# How to Run the Script Inside Docker
# ====================================
# Prerequisites:
#   - Docker Desktop installed and running
#   - case3.py, sample.csv, and Dockerfile are in the same directory

# Step 1: Open terminal/PowerShell and navigate to the directory
cd /path/to/your/files

# Step 2: Build the Docker image
#   -t case3 assigns the name 'case3' to the image
docker build -t case3 .
```

```
# Step 3: Run the container
#   -v mounts your local directory to /app/output inside the container
#   so the generated CSV files are accessible on your machine after the run
docker run -v $(pwd)/output:/app case3

# Step 4: Check the output
#   The 3 CSV files will appear in your current directory:
#     sample_output_5.csv
#     sample_output_7.csv
#     sample_output_10.csv

# Note for Windows PowerShell users:
#   Replace $(pwd) with ${PWD}:
docker run -v ${PWD}:/app case3
```